SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom' s Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

I AATHISRI P(192521252) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
AATHISRI P(192521252)

1. Error Analysis in Equivalence Class Partitioning and Boundary Value Analysis

Introduction

Equivalence Class Partitioning (ECP) and Boundary Value Analysis (BVA) are black-box testing techniques used to reduce the number of test cases while detecting input-related defects.

Errors in the test design

- Valid and invalid inputs are incorrectly grouped into the same equivalence class.
- The tester does not identify separate classes for different input conditions.
- Boundary values are ignored, even though errors frequently occur at the limits.
- The test cases do not adequately cover minimum, maximum, and just-outside-limit values.

Root causes

These errors occur because the tester does not understand the input specifications properly and assumes that testing representative values alone is sufficient. Random or poorly selected values may miss validation errors.

Corrected test cases

Assume the shopping system accepts a quantity from 1 to 100 items.

Equivalence class	Test input	Expected result
Invalid: below minimum	0	Reject
Valid: minimum quantity	1	Accept
Valid: normal quantity	50	Accept
Valid: maximum quantity	100	Accept
Invalid: above maximum	101	Reject
Invalid: negative quantity	-1	Reject

Invalid: non-numeric input	abc	Reject

Boundary values should include 0, 1, 2, 99, 100, and 101.

Improved testing approach

1. Study the input requirements and identify the valid range.
2. Divide inputs into separate valid and invalid equivalence classes.
3. Select representative values from each class.
4. Test values at and around both boundaries.
5. Compare actual results with expected results.

Conclusion

Correct ECP reduces redundant test cases, while BVA detects off-by-one errors and incorrect validation. Using both techniques improves defect detection and the reliability of the shopping system.

2. Error Analysis of Random Testing in a Banking Application

Introduction

Random testing selects inputs without following a structured test design. Although it can discover unexpected defects, it does not guarantee adequate coverage of important requirements or boundary conditions.

Errors in the testing strategy

- Test inputs are selected without systematic planning.
- Transaction minimums and maximums may not be tested.
- Invalid transactions and exceptional conditions may be overlooked.
- The tester assumes that many random tests automatically provide high coverage.
- There is no clear mapping between requirements and test cases.

Root-cause analysis

Random testing may repeatedly select ordinary transaction amounts while rarely selecting critical boundary values. Consequently, defects such as accepting an amount above the permitted limit or rejecting a valid minimum transaction may remain undetected.

Corrected black-box testing techniques

Assume a banking application allows transactions from ₹100 to ₹50,000.

Technique	Test input	Expected result
Equivalence partitioning	₹5,000	Accept valid amount
Equivalence partitioning	₹50	Reject below minimum
Equivalence partitioning	₹60,000	Reject above maximum
Boundary value analysis	₹99	Reject
Boundary value analysis	₹100	Accept
Boundary value analysis	₹101	Accept

Boundary value analysis	₹49,999	Accept
Boundary value analysis	₹50,000	Accept
Boundary value analysis	₹50,001	Reject
Negative testing	Text instead of amount	Reject

These limits are illustrative; actual test cases must use the application's specified transaction limits.

Improved strategy

1. Derive test cases from banking requirements.
2. Use ECP to identify valid and invalid transaction classes.
3. Use BVA to test the minimum and maximum limits.
4. Test zero, negative, malformed, and unauthorized transactions where applicable.
5. Use random testing as an additional technique rather than the only technique.
6. Record test results and trace them to requirements.

Conclusion

Random testing alone cannot guarantee adequate coverage. Combining structured black-box techniques with selected random tests improves reliability and helps identify transaction-limit defects before deployment.

3. Error Analysis in Control Flow Graph and Path Testing

Introduction

White-box testing examines the internal structure of a program. A Control Flow Graph (CFG) represents program execution using nodes and directed edges. Nodes represent statements or blocks, while edges represent possible control transfers.

Errors in the CFG

- Decision nodes for if, if-else, and other conditions are missing.
- Loop entry, loop body, and loop exit paths are not represented.
- Some execution paths are omitted.
- Independent paths are not identified.
- Test cases are designed without checking branch or path coverage.

Root causes

The developer may have simplified the program structure or failed to examine every decision and loop. As a result, the CFG does not accurately represent the program's possible execution paths.

Corrected CFG example

Consider the following pseudocode:

```
Start
  |
Read x
  |
x > 0?
 /   \
Yes   No
|     |
Print   Print
Positive Non-positive
 \     /
   End
```

The decision x > 0 must have two outgoing edges: True and False.

Independent paths

- Path 1: Start → Read x → x > 0 True → Print Positive → End.

- Path 2: Start → Read x → x > 0 False → Print Non-positive → End.

Both paths require at least one test case.

For example:

Test case	Input	Path exercised	Expected output
TC1	5	True branch	Positive
TC2	0	False branch	Non-positive

Cyclomatic complexity

Cyclomatic complexity estimates the number of linearly independent paths in a connected CFG:

\[V(G)=E-N+2P \]

Where:

- \(E\) = number of edges
- \(N\) = number of nodes
- \(P\) = number of connected components, usually 1

For a single connected graph, the formula becomes \(V(G)=E-N+2\).

Improved testing approach

1. Identify all statements, decisions, loops, and exits.
2. Construct the complete CFG.
3. Calculate cyclomatic complexity.
4. Identify independent paths.
5. Create test cases to execute those paths.
6. Measure branch and path coverage.

Conclusion

A complete CFG helps identify missing execution paths. Independent path testing improves structural coverage and exposes logical errors, incorrect decisions, and loop-related defects.

CSA37-ST_CO2_AT3_Error Analysis Task.docx.docx

4. Why 100% Statement Coverage Is Not Enough

Introduction

Statement coverage measures the percentage of executable statements that are executed during testing. Achieving 100% statement coverage does not guarantee that every decision outcome has been tested.

Errors in the testing approach

- The testing team focuses only on executing statements.
- Both True and False outcomes of conditions may not be tested.
- Compound conditions may contain logical errors.
- Error-handling branches may remain untested.
- The team incorrectly assumes statement coverage proves complete testing.

Example

Consider the following code:

```
if (age >= 18)
   print("Eligible");
else
   print("Not eligible");
```

Test cases:

Test input	Expected result	Branch
Age = 20	Eligible	True
Age = 16	Not eligible	False
Age = 18	Eligible	True

Testing with ages 20 and 16 executes both statements and covers both branches. The additional age-18 test checks the eligibility boundary.

For more complex conditions, statement coverage may be insufficient even when every statement has executed.

Improved white-box testing strategies

1. Branch coverage: Test every possible decision outcome, including True and False branches.

2. Condition coverage: Ensure each individual Boolean condition evaluates to both True and False at least once.

3. Multiple-condition testing: For compound decisions, test combinations of condition values where practical.

4. Path coverage: Test independent execution paths through the program. Complete path coverage may be impossible when loops create infinitely many paths.

Advantages

- Detects incorrect decision logic.
- Identifies untested error-handling paths.
- Improves confidence in program behaviour.
- Provides stronger evidence of testing adequacy than statement coverage alone.

Conclusion

Statement coverage is useful but limited. Combining it with branch coverage, condition coverage, and selected path testing improves defect detection and makes testing more effective.

5. Correcting Equivalence Classes for Age Validation

Introduction

The age-validation field accepts ages from 0 to 100. The tester creates only two classes: negative numbers and positive numbers. This classification is incomplete because positive numbers include both valid ages and values greater than the permitted maximum.

Errors identified

- All positive values are treated as one class.
- Valid ages are not separated from invalid ages above 100.
- The upper boundary is ignored.
- Values such as decimals, text, and empty input may not be considered.
- Boundary Value Analysis is not performed.

Corrected equivalence classes

Class	Input range/example	Expected result
Invalid: negative age	-1	Reject
Valid: lower boundary	0	Accept
Valid: normal age	25	Accept
Valid: upper boundary	100	Accept
Invalid: above maximum	101	Reject
Invalid: non-numeric input	abc	Reject
Invalid: missing input	Empty	Reject if required

Boundary Value Analysis

For the valid range of 0–100, test these values:

Boundary test	Input	Expected result
Just below minimum	-1	Reject
Minimum	0	Accept
Just above minimum	1	Accept
Just below maximum	99	Accept
Maximum	100	Accept
Just above maximum	101	Reject

Root causes

The tester did not analyse the complete input specification and treated numerical positivity as equivalent to validity. The absence of boundary tests also allows off-by-one errors to remain undetected.

Conclusion

Correct equivalence classes distinguish valid values from every important invalid category. BVA verifies the limits and values immediately outside them, improving input validation and reducing age-related defects.

6. Error Analysis of Condition Coverage

Note: Question 6 in the uploaded document is incomplete. The visible text mentions a developer claiming full Condition Coverage for a complex if(A && B ... expression. The answer below addresses that visible topic.

Introduction

Condition coverage is a white-box testing technique that checks whether each individual Boolean condition in a decision evaluates to both True and False at least once.

Consider the example:

```
if (A && B)
   print("Accepted");
else
   print("Rejected");
```

Here, A and B are individual Boolean conditions.

Errors to examine

- The developer may confuse condition coverage with decision coverage.
- Testing only cases where the complete expression is True does not test all condition outcomes.
- Compound conditions may not have all relevant combinations tested.
- Short-circuit evaluation may prevent B from being evaluated when A is False.
- The claim of full coverage may be unsupported by a condition-by-condition test record.

Corrected test cases

Test case	A	B	Decision result
TC1	True	True	True
TC2	True	False	False
TC3	False	True	False
TC4	False	False	False

These four tests cover all combinations of two Boolean conditions. They also ensure each condition takes both True and False values across the test set.

Root-cause analysis

A tester may observe that the decision was executed without checking the individual condition outcomes. In compound expressions, the result of the entire expression can be False for several different reasons, so checking only the final result may hide defects.

Improved testing approach

1. Identify every individual condition in the decision.
2. Design tests that make each condition True and False.
3. Test important combinations of conditions.
4. Verify both True and False decision outcomes.
5. Record condition and branch coverage separately.
6. For safety-critical logic, consider Modified Condition/Decision Coverage (MC/DC), where each condition is shown to independently affect the decision result.

Conclusion

Condition coverage helps reveal errors hidden within compound Boolean expressions. Combining condition coverage with branch coverage and, when appropriate, MC/DC provides stronger evidence that decision logic has been tested adequately.

7. Correcting Decision Table Testing for a Discount System

Introduction

Decision Table Testing is a black-box technique used when output depends on combinations of conditions. In a discount system, the conditions may include whether a customer is a premium member and whether a coupon has been applied.

Errors in the existing decision table

- Some combinations of conditions are missing.
- Premium membership and coupon status may not be evaluated independently.
- Overlapping discount rules may be handled incorrectly.
- The expected discount for each combination is not clearly defined.
- Test cases may cover individual conditions but miss their combinations.

Corrected decision table

Assume the following illustrative business rules:

- Premium members receive a 10% discount.
- Customers using a valid coupon receive a 5% discount.
- If both apply, the discounts are combined additively for this example.

Rule	Premium member?	Coupon applied?	Expected discount
R1	Yes	Yes	15%
R2	Yes	No	10%
R3	No	Yes	5%
R4	No	No	0%

The percentages and combination rule are examples. The actual table must follow the application's stated discount policy.

Test cases

Assume the original price is ₹1,000.

Test	Premium	Coupon	Expected final price
TC1	Yes	Yes	₹850
TC2	Yes	No	₹900
TC3	No	Yes	₹950
TC4	No	No	₹1,000

Root-cause analysis

The testing team may have created an incomplete table or failed to enumerate all combinations. This allows incorrect pricing calculations to remain undetected, especially when both conditions apply simultaneously.

Improved testing approach

1. List all relevant conditions and possible values.
2. Enumerate every meaningful combination.
3. Define the expected action or discount for each rule.
4. Create at least one test for each rule.
5. Check boundary prices, invalid coupons, and discount restrictions where specified.
6. Compare calculated prices with expected results.

Conclusion

An exhaustive decision table provides systematic coverage of combinations and clarifies business rules. It reduces missed scenarios and helps prevent incorrect discounts and production pricing defects.

8. Error Analysis in Loop Testing

Introduction

Loop Testing is a white-box testing technique used to verify the correctness of loops, including their initialization, termination condition, and number of iterations.

The tester checks only the first iteration (\(n=1\)) and the maximum iteration (\(n=\text{max}\)). This misses important cases such as zero iterations and one iteration beyond the permitted maximum.

Errors identified

- The zero-iteration case (\(n=0\)) is not tested.
- The invalid case (\(n=\text{max}+1\)) is ignored.
- Loop termination conditions may contain off-by-one errors.
- The loop body may fail when it is skipped.
- Nested loops may have interaction defects that are not detected.

Root causes

The tester assumes that the first and last valid iterations are sufficient. However, defects can occur before the loop begins, at the termination boundary, or when the loop executes too many times.

Corrected test cases

Assume a loop permits between 1 and 10 iterations.

Test case	Iterations	Purpose
TC1	0	Check zero-iteration handling
TC2	1	Check minimum valid iteration
TC3	2	Check a small number of iterations
TC4	9	Check near-maximum iterations
TC5	10	Check maximum permitted iterations
TC6	11	Check behaviour beyond the maximum

The expected result for 0 and 11 depends on the specification. The program should either handle zero iterations correctly or reject an invalid count, and it should prevent execution beyond the allowed maximum.

Testing different loop structures

1. Simple loops: Test zero iterations, one iteration, two iterations, a typical count, maximum iterations, and maximum plus one.

2. Nested loops: Test the inner loop at its minimum, typical, and maximum counts while controlling the outer loop. Then vary the outer loop and test important combinations.

3. Concatenated loops: Test each loop independently and then test their combined execution.

4. Unstructured loops: If practical, redesign the loop into a simpler structure before testing.

Conclusion

Testing only the first and last valid iterations is insufficient. Systematic loop testing checks initialization, termination, boundary conditions, and nested-loop interactions, improving path integrity and reducing loop-related defects.

9. Error Analysis of Error Guessing in a Safety-Critical Braking System

Introduction

Error Guessing is a testing technique in which the tester uses experience to predict where defects are likely to occur. It is useful as a supplementary technique but is insufficient as the only method for a safety-critical braking system.

Errors in the strategy

- Testing depends heavily on individual experience.
- Test cases may not systematically cover all input boundaries.
- Important braking-system states and transitions may be overlooked.
- Test completeness is difficult to measure.
- Safety requirements may not be traced to test cases.
- Rare but dangerous failures may remain undetected.

Root causes and impact

Error Guessing is ad hoc, so two testers may choose different test cases. A tester might check normal braking but overlook sensor failure, sudden changes in speed, or invalid sensor values. Missing these cases can lead to unsafe braking behaviour.

Corrected testing strategy

1. Boundary Value Analysis (BVA)

Test values around specified speed, pressure, temperature, and sensor limits.

For example, if a braking function has a specified activation threshold of 80 km/h, relevant tests may include:

Test input	Purpose
79 km/h	Below threshold
80 km/h	At threshold
81 km/h	Above threshold

The expected braking response must come from the system's actual safety requirements; the example does not define a real braking rule.

2. State Transition Testing

Model the important system states, such as:

- Normal operation
- Braking requested
- Braking active
- Sensor fault
- Fail-safe mode

Test valid transitions, such as normal operation to braking active, and fault transitions, such as sensor failure to fail-safe mode, wherever required by the specification.

3. Negative and fault testing

Test invalid sensor readings, communication loss, delayed responses, and recovery from faults.

4. Error Guessing

Retain experience-based tests to supplement systematic coverage.

Advantages

- BVA detects errors at critical limits.
- State Transition Testing checks behaviour across operational states.
- Fault testing evaluates the system's response to abnormal conditions.
- Requirement traceability provides evidence of test coverage.

Conclusion

For a safety-critical braking system, Error Guessing should not be used alone. Combining BVA, State Transition Testing, fault testing, and requirement-based testing provides stronger safety assurance. Testing supports safety verification but does not, by itself, prove that the system is completely safe.

10. Correcting a Control Flow Graph with Missing else Branches

Introduction

A Control Flow Graph (CFG) represents all possible control transfers in a program. Each decision must include its possible outcomes. If nested if statements omit their else branches, the CFG becomes incomplete and error paths may remain untested.

Errors in graph construction

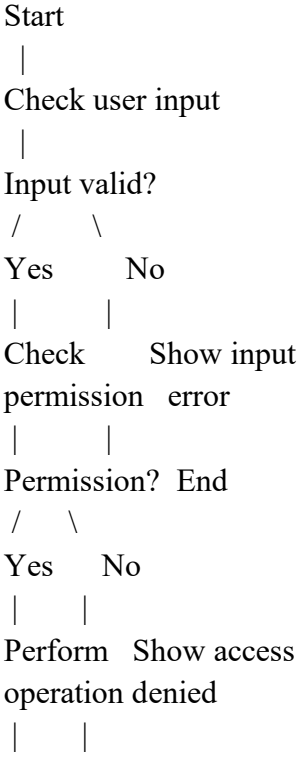
- False outcomes of decisions are missing.
- Error-handling statements are not represented.
- Nested decisions are simplified incorrectly.
- Independent paths are not identified.
- Statement coverage is incorrectly treated as proof of branch coverage.

Root-cause analysis

The tester may have created the CFG using only the normal execution path. This allows all normal statements to execute while the False branches and error-handling paths remain untested. Consequently, the graph does not represent every possible execution route.

Corrected example

Consider the following pseudocode:



End End

The corrected CFG includes both outcomes of each decision:

- Valid input or invalid input.
- Permission granted or permission denied.

Independent paths

Path 1: Successful execution

Start → Check input → Valid → Check permission → Granted → Perform operation → End.

Path 2: Invalid input

Start → Check input → Invalid → Show input error → End.

Path 3: Permission denied

Start → Check input → Valid → Check permission → Denied → Show access denied → End.

These paths cover the normal route and both error-handling routes in this example.

Test cases

Test case	Input condition	Permission	Expected result
TC1	Valid	Granted	Operation performed
TC2	Invalid	Not applicable	Input error displayed
TC3	Valid	Denied	Access denied message

Improved testing approach

1. Review the source code and identify every decision.
2. Add True and False edges for each decision.
3. Represent nested decisions and error-handling statements.
4. Identify independent paths using the completed CFG.
5. Create tests for normal and error paths.
6. Measure branch coverage separately from statement coverage.

Conclusion

A complete CFG must represent all decision outcomes, including nested else branches. Testing independent paths improves branch adequacy, verifies error handling, and helps uncover defects that statement coverage alone may miss.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand